

CSCI 361 Lecture 10: Non Context-Free Languages

Shikha Singh

Announcements & Logistics

- **HW 5** is out, due next Wed at 10 pm
- Questions on CFGs and PDAs
- No office hours on Tues Oct 14, instead:
 - Today: 1-2.30 pm
 - Friday: 10-11 am
- No class next Tuesday due to reading period
- Pick up **Exercise # 8** due next Thursday
- Midterm 1 has been graded: discuss in last 10 mins

Last Time

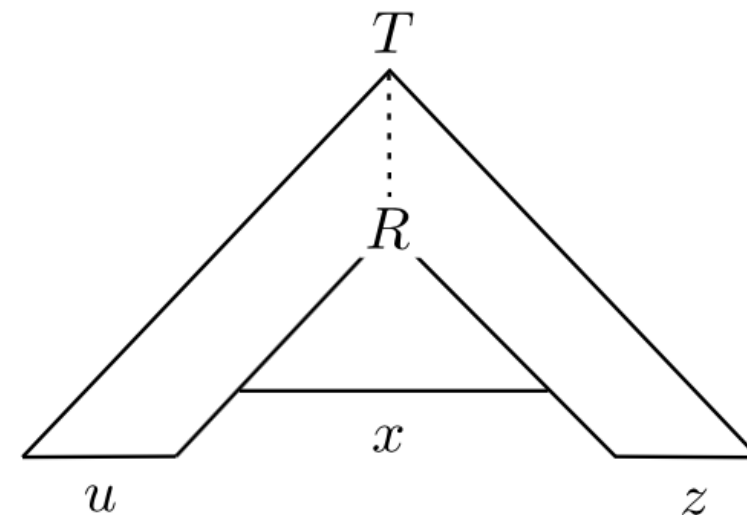
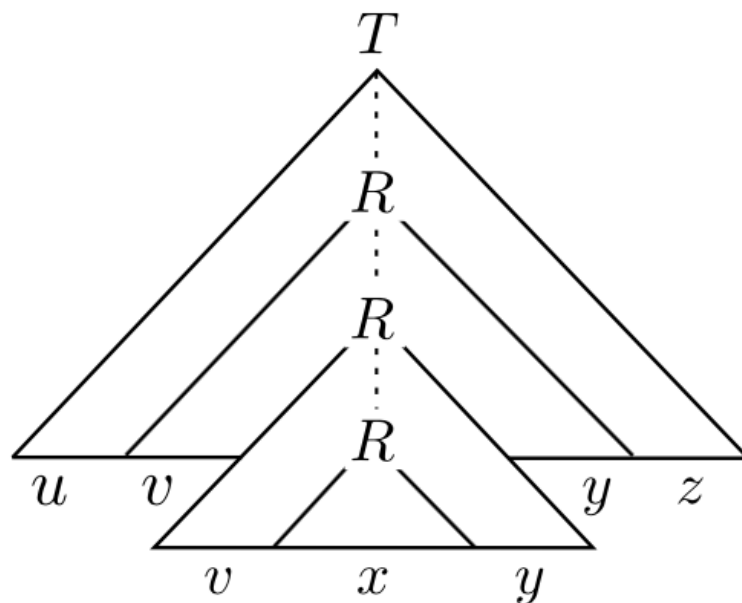
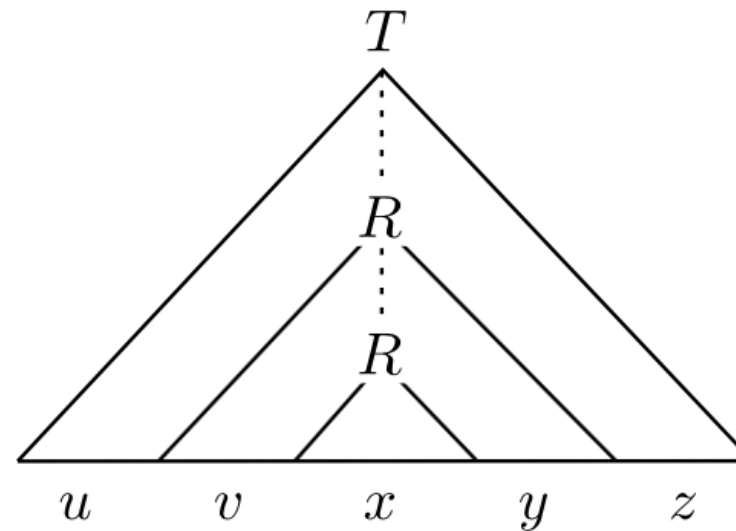
- Discussed Pumping lemma for CFLs
- More examples today
- Wrap up some remaining loose ends about CFLs
- Start Turing Machines

Pumping Lemma: CFLs

- **Statement:** If L is a CFL, then there exists a number p (the pumping length) such that for all $s \in L$ of length at least p , it is possible to divide s into five pieces $s = uvxyz$ satisfying the conditions
 1. $|vy| > 0$
 2. $|vxy| \leq p$
 3. For each $i \geq 0$, $uv^i xy^i z \in L$
- Note that vxy can appear anywhere in the string as long as they are no longer than p symbols long

Pumping Lemma: CFLs

- **Main idea:** If a string has length at least $p = 2^{|V|} + 1$ then some variable is repeated in the parse tree



Recap: Using the Pumping Lemma

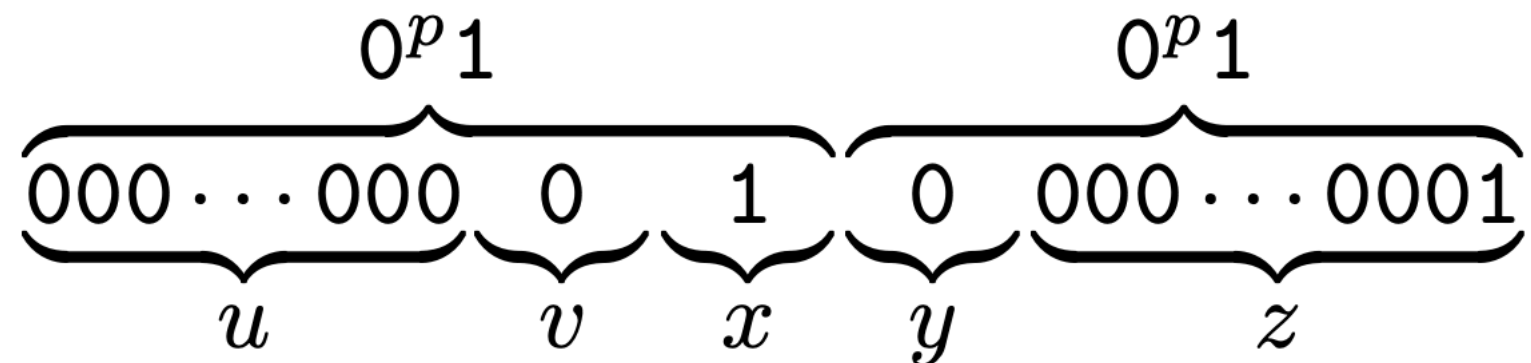
- **Problem.** Apply the pumping lemma to prove that the language $\{a^n b^n c^n \mid n \geq 0\}$ is not context-free.
- Proof. Assume L is context-free with pumping length p .
- Select $w = a^p b^p c^p \in L$ and has length $3p \geq p$
- Consider all possible ways to partition w into $uvxyz$ s.t. condition (2) and (3) hold: $|vy| > 0$ and $|vxy| \leq p$
 - **Case 1.** At least one of v or y contains two distinct symbols. Then xv^2xy^2z contains symbols out of order and $\notin L$
 - **Case 2.** Both v and y contain the same symbol (both are a 's or both b 's or both c 's then $uxz \notin B$

Practice

- **Problem.** Apply the pumping lemma to prove that the language $\{ww \mid w \in \{0,1\}^*\}$ is not context-free.

Practice

- **Problem.** Apply the pumping lemma to prove that the language $\{ww \mid w \in \{0,1\}^*\}$ is not context-free.
- Choosing $w = 0^p 1 0^p 1$ does not work



- Matching two different "pairs" of strings is essence of being non CFL
- Another try to capture this: $w = 0^p 1^p 0^p 1^p$

Pumping Lemma Proof Tips

- Proofs using the PL for CFLs devolve to examining a bunch of cases
- Try to select w that will lead to as few cases as possible
- Try to cover as many similar cases at once as possible: if several cases are analogous, address them in one general argument
- Try to use closure properties whenever possible

CFL: Intersection Closure

- **Theorem.** If C is a context-free language and R is a regular language then $L \cap R$ is context-free.
- Proof Idea.
 - P be a PDA that recognizes C and M be DFA that recognizes R
 - Let Q, Q' be the set of states of P, M , create a new PDA P' with states $Q \times Q'$
 - P' simulates P as well as M and accepts a string if both accept
 - Ignores P 's stack on M 's transitions, just remembers states of M

CFL: Intersection Closure

- **Note.** Intersection of two CFLs is not necessarily context-free!
 - Example?

Context-Free or Not?

- **Question.** One of these languages is CF and the other is not, can you identify which is which?
 - $L_1 = \{w a^n b^n w^R \mid w \in \{a, b\}^*, n \geq 0\}$
 - $L_2 = \{w a^n w^R b^n \mid w \in \{a, b\}^*, n \geq 0\}$

Context-Free or Not?

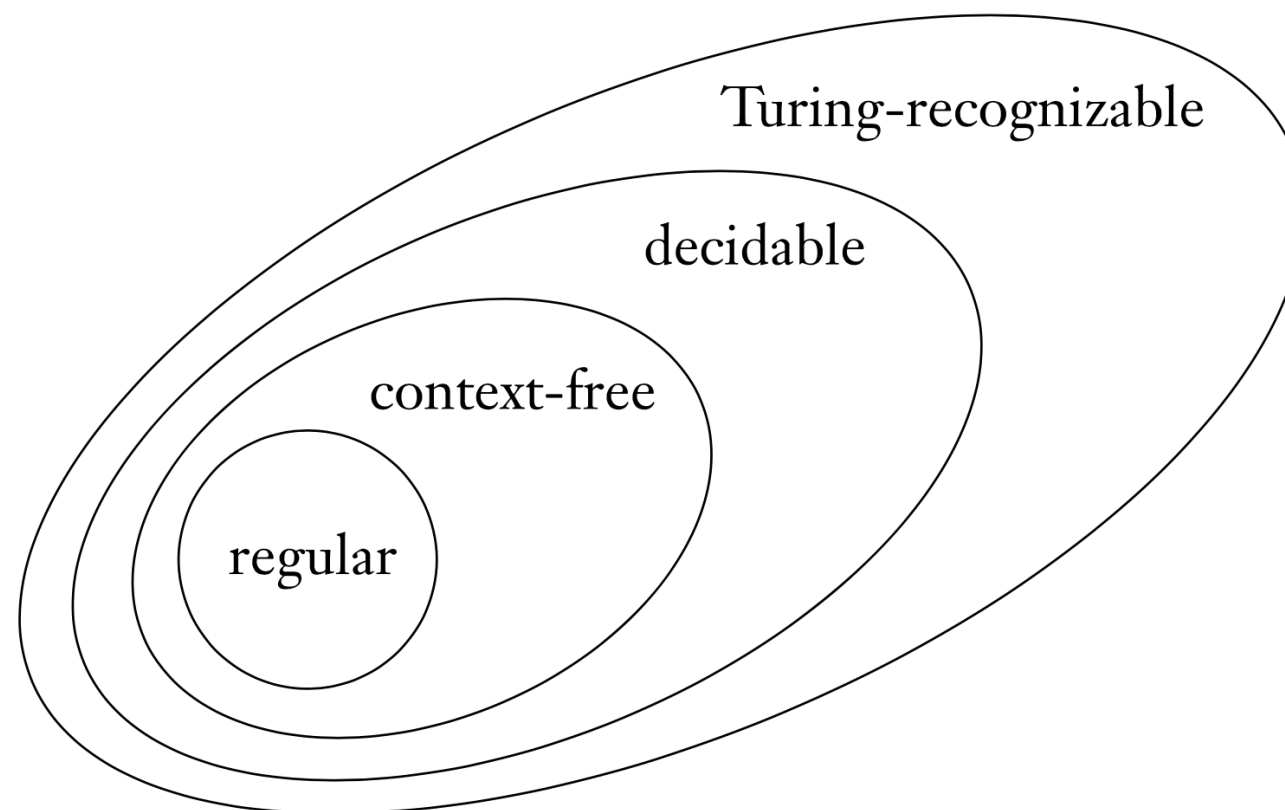
- $L_1 = \{w a^n b^n w^R \mid w \in \{a, b\}^*, n \geq 0\}$
- $L_2 = \{w a^n w^R b^n \mid w \in \{a, b\}^*, n \geq 0\}$
- **Answer.** L_1 is context-free but L_2 is not.
- Intuition: need to match two "pairs": can do it if they are next to each other but not if they are separated
- CFG for L_1 ?
 - $S \rightarrow aSa \mid bSb \mid A$
 - $A \rightarrow aAb \mid \varepsilon$
- **Exercise.** Can show L_2 is not CF using the pumping lemma, use $w = b^p a^p b^p b^p$

Examples of Non CFLs

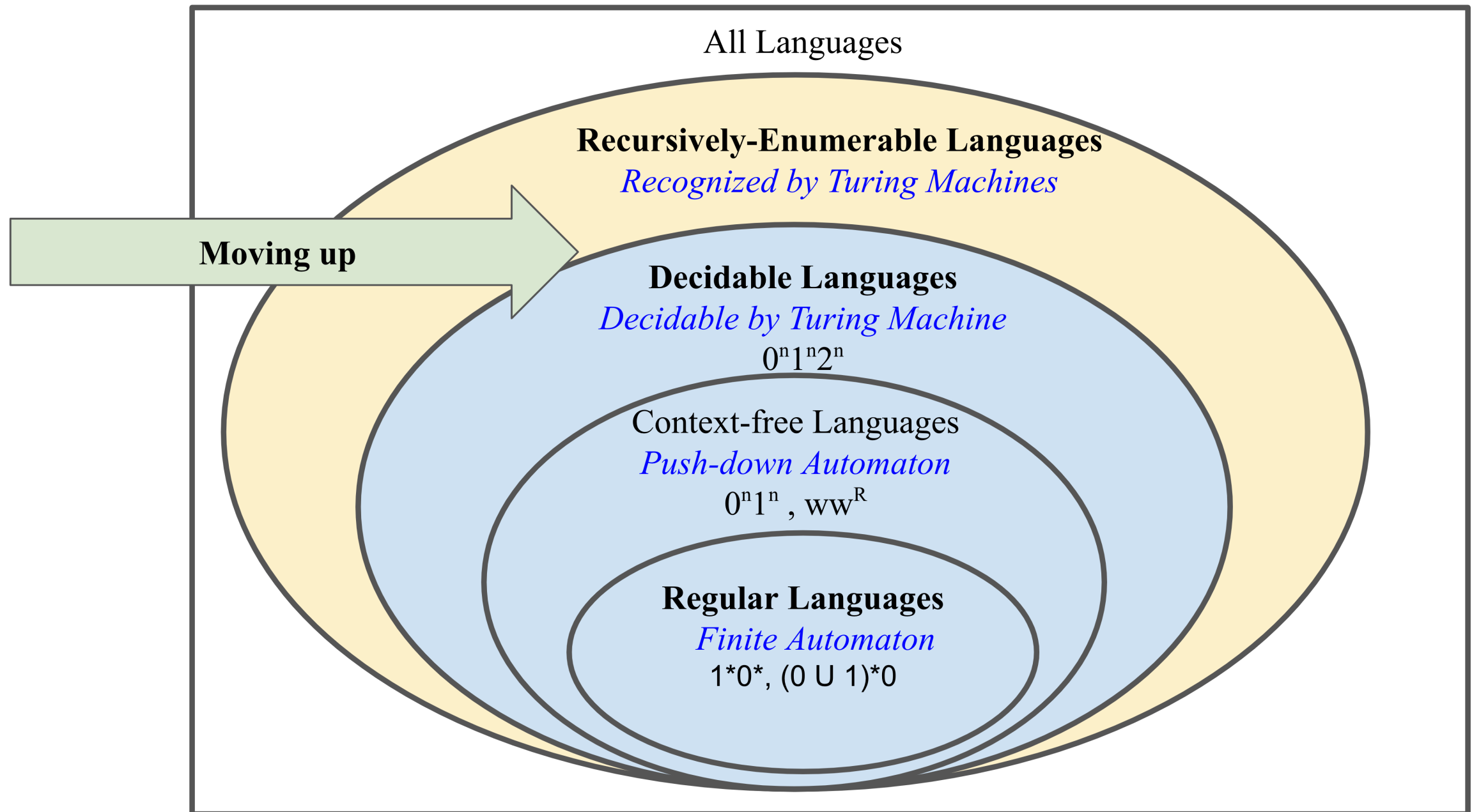
- Pairing/Counting examples we have seen:
 - $\{a^n b^n c^n \mid n \geq 0\}$, $\{a^n b^n a^n\}$, $\{ww \mid w \in \{a, b\}^*\}$
 - HW: language of palindromes with equal # of 1s and 0s
 - Strings over $\{a, b, c\}$ with equal # of a's, b's and c's
 - $\{a^n b^m a^n b^m \mid n, m \geq 0\}$
 - $\{w a^n w^R b^n \mid w \in \{a, b\}^*, n \geq 0\}$
- Non-linear counting examples:
 - $\{a^{2^n} \mid n \geq 0\}$, $\{a^p \mid p \text{ is a prime}\}$, $\{a^{n^2} \mid n \geq 0\}$
 - Intuition: structure is too rigid to be able to be "pumped"

Models of Computation vs Power

- Finite automaton
- Push-down automaton
- Turing machine
- Sequencing, repetition
- Function calls/ simple recursion
- Simple loops as well as multiple recursive calls

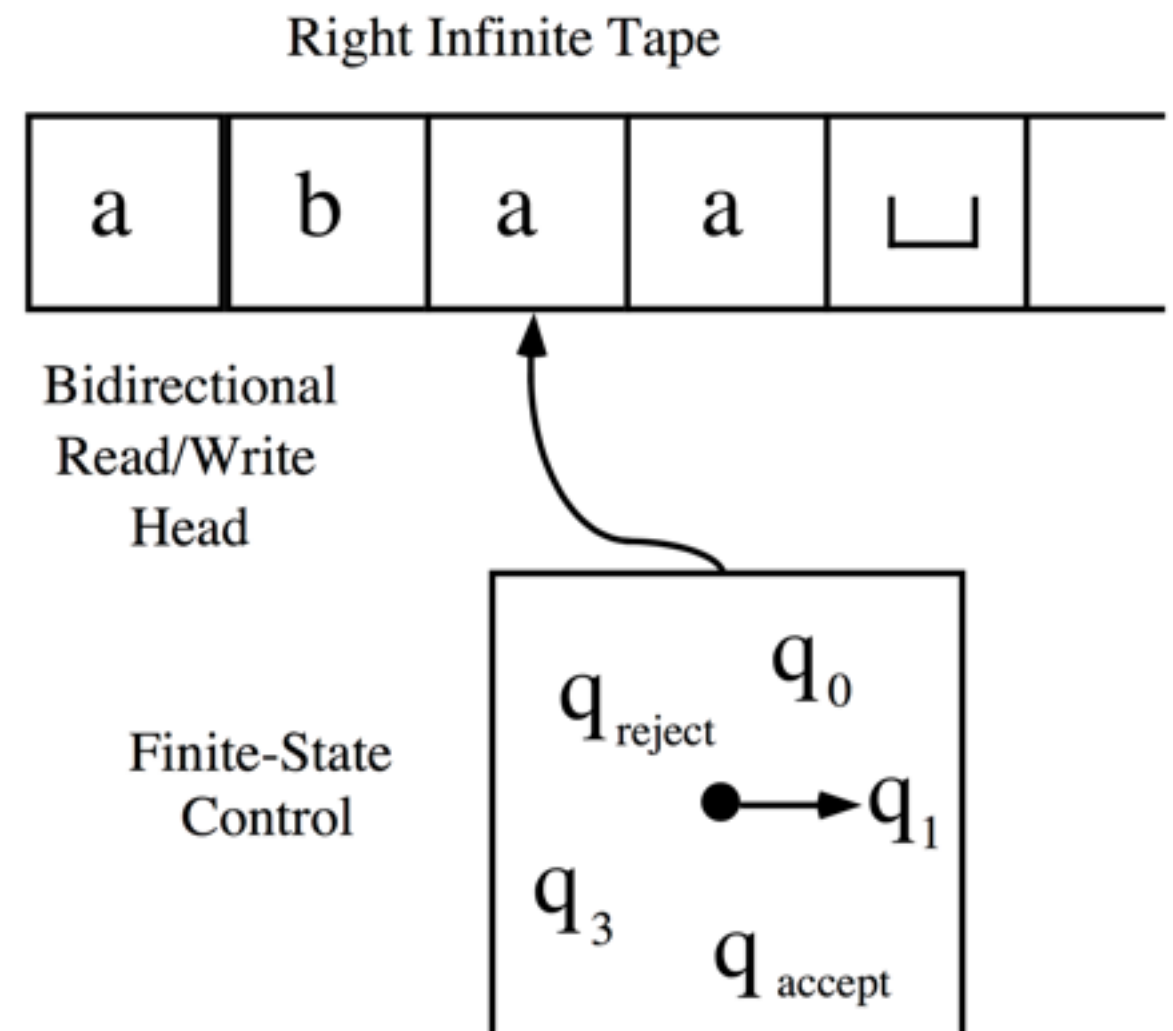


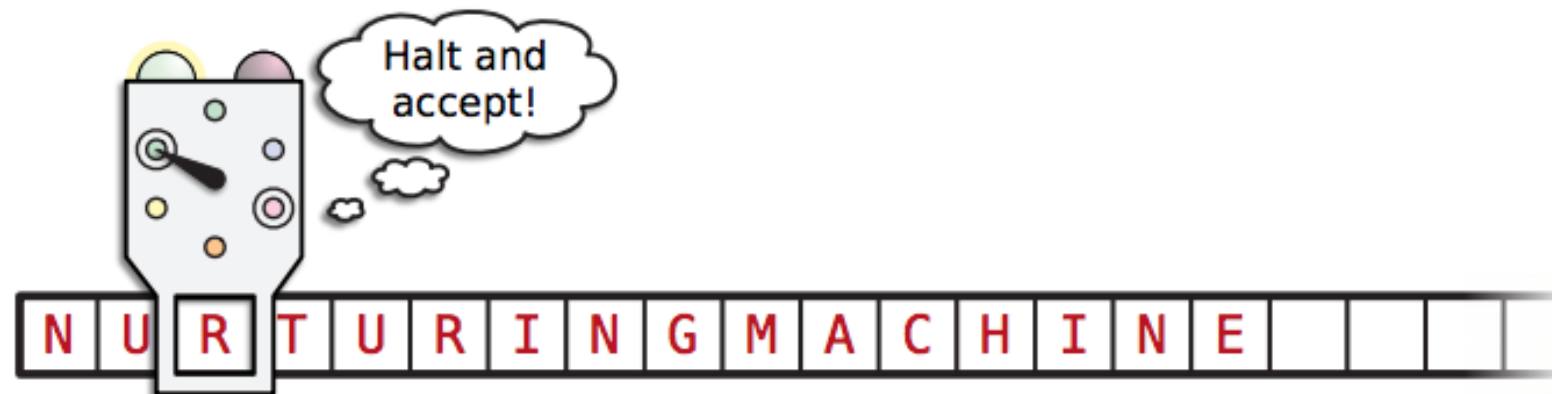
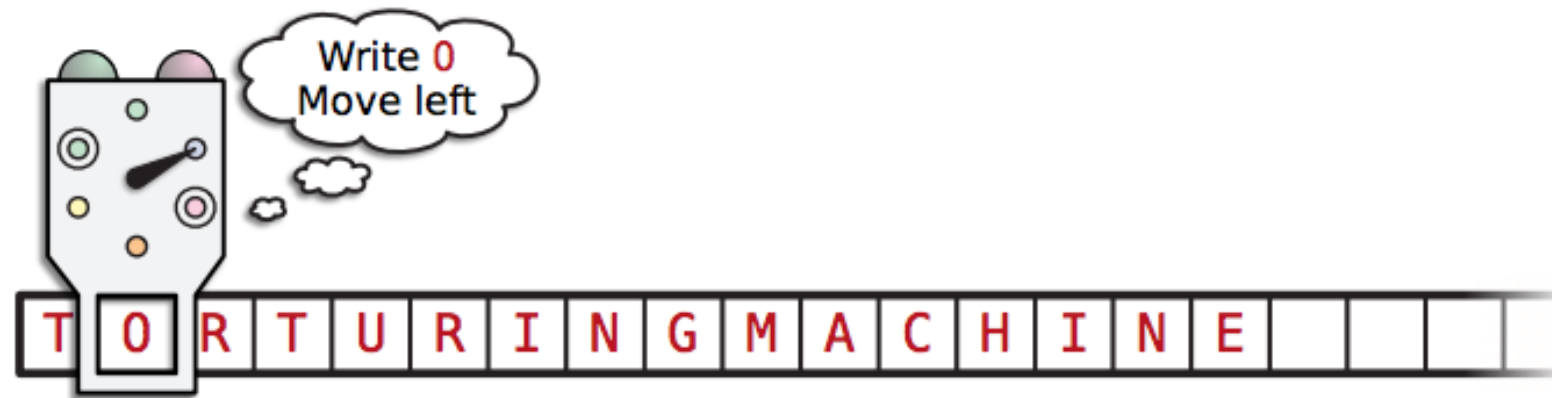
Moving Up



Turing Machines

- Finite number of states
- Infinite tape (memory)
- Read-write head that can move right and left on the tape
- Can modify the input
- Special accept/reject states





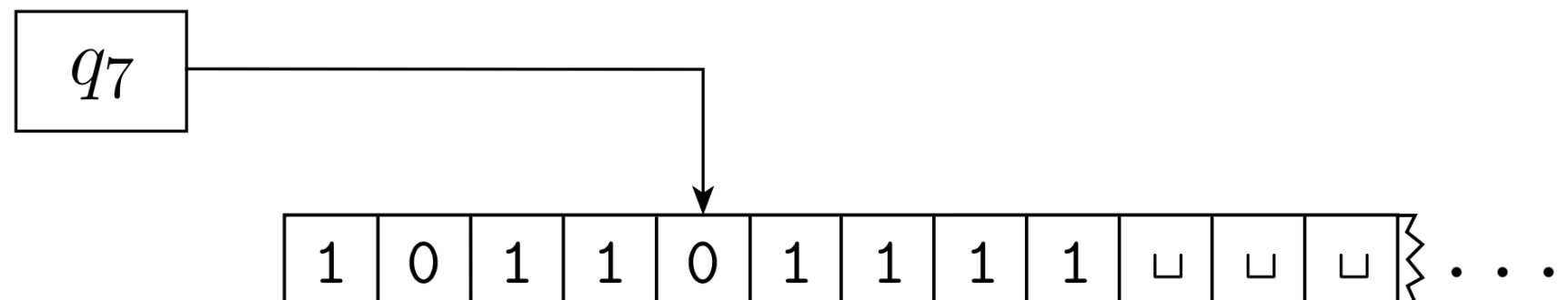
A few iterations of a six-state Turing machine.

Formal Definition

- A Turing Machine is a 7-tuple $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$, where Q, Σ, Γ are all finite sets
- Q is the set of **states**
- Σ is the **input alphabet** and does not contain the **blank symbol** $\sqcup$
- Γ is the **tape alphabet** where $\sqcup \in \Gamma$ and $\Sigma \subset \Gamma$
- $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ is the **transition function**
- $q_0, q_{\text{accept}}, q_{\text{reject}} \in Q$ are the **start**, **accept** and **reject** states where $q_{\text{accept}} \neq q_{\text{reject}}$

How a TM Computes

- Initially, input $w = w_1w_2\cdots w_n \in \Sigma^*$ on the leftmost n squares, rest has $\sqcup$ and **head** of the TM in the leftmost position
- The computation proceeds using δ : can move left or right, alter tape contents and change states
- Configuration** of a TM: current state, tape contents & head location
 - Written as uqv : Current state is q , current tape contents is uv , current head location is first symbol of v

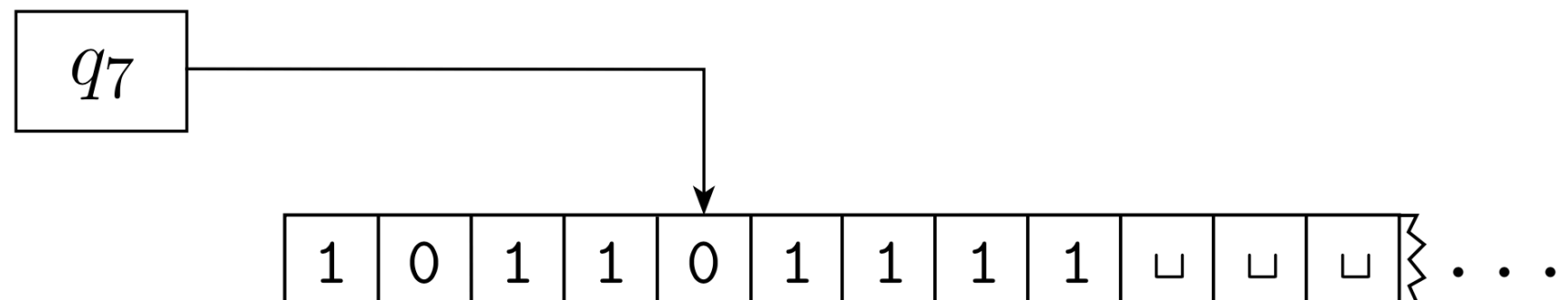


How a TM Computes

- A configuration C_1 **yields** a configuration C_2 if the TM can legally go from C_1 to C_2 using its transition function
- Consider symbols $a, b, c \in \Gamma$ and strings $u, v \in \Gamma^*$ then

$ua q_i bv$ yields $u q_j acv$ if $\delta(q_i, b) = (q_j, c, L)$, and

$ua q_i bv$ yields $uac q_j v$ if $\delta(q_i, b) = (q_j, c, R)$



Language of a TM

- **Start** configuration: q_0w
- **Accepting** configuration if the current state is q_{accept}
- **Rejecting** configuration if the current state is q_{reject}
- A TM M **accepts** an input w if a sequence of configurations $C_1, \dots, C_k$ exist such that
 - C_1 is the start configuration, each C_i yields C_{i+1} and C_k is an accepting configuration
- The set of strings accepted by M is the language **recognized** by M , denoted $L(M)$

Turing Machine Loops

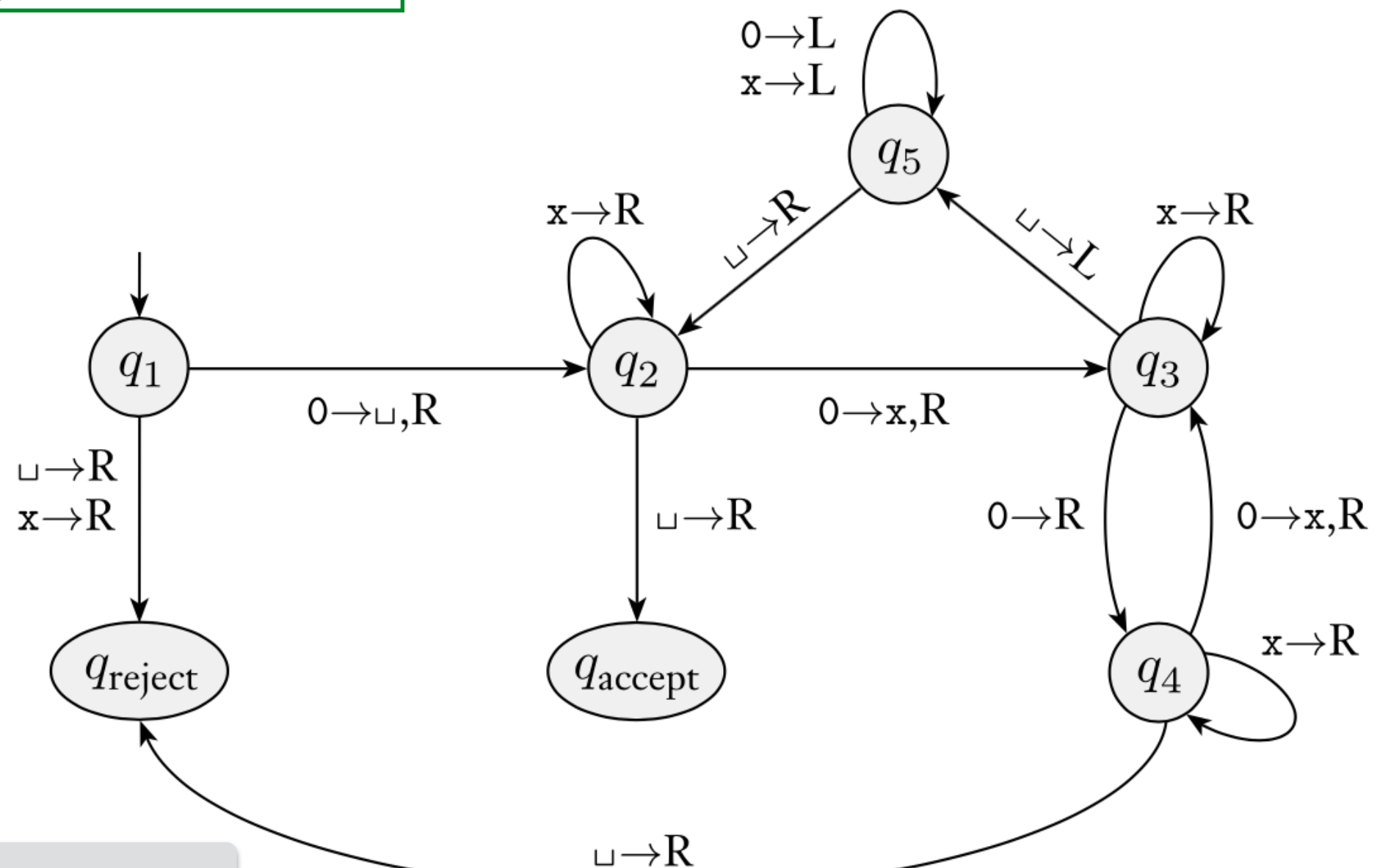
- An important distinction between DFA/PDA and a TM
- On an input w , a TM can:
 - Accept w (and halt)
 - Reject w (and halt)
 - "Loop" on an input w (never halt): *this is new!*
- **Definition (Decidable).** A language L is **TM-decidable** or decidable if there is a TM that accepts every string in L and rejects every string not in L (i.e., it halts on all inputs in Σ^*)
 - A TM is **decider** if it halts on every input in Σ^*

Turing Machine Languages

- **Definition (TM Recognizable).** A language L is **TM-recognizable** if there is a TM that accepts every string in L and loops or rejects strings not in L .
 - $w \in L \iff$ TM halts and accepts
 - But if $w \notin L$, TM may never halt
- **Definition (TM Decidable).** A language L is **TM-decidable** or decidable if there is a TM that accepts every string in L and rejects every string not in L (i.e., it halts on all inputs in Σ^*)
 - A TM is **decider** if it halts on every input in Σ^*

- **Example TM:** Consider a TM for the language $A = \{0^{2^n} \mid n \geq 0\}$

Each transition of the form $x \rightarrow y, D$ means “upon reading x , replace it with symbol y and move the tape head in direction D ”. If y is omitted x is left unchanged



State diagram: low-level description

Medium-Level Description

Consider a TM M for the language $A = \{0^{2^n} \mid n \geq 0\}$:

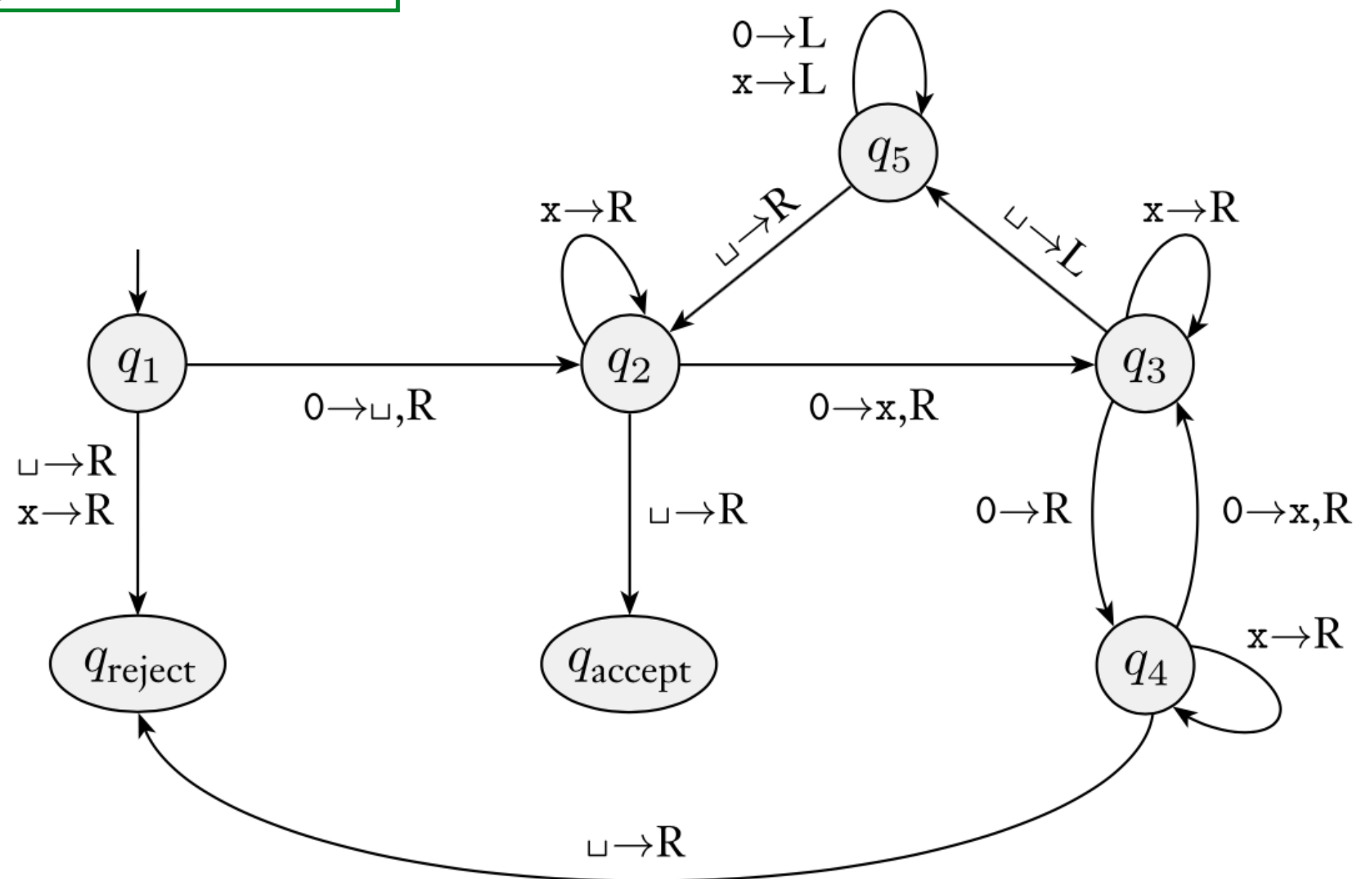
$M =$ "On input string w ,

1. Sweep left to right across the tape, crossing off every other zero.
2. If in Stage 1 and there is a single zero, **accept**
3. If in Stage 1 and there are more than one odd zeros, **reject**
4. Return to the lefthand end of tape and go to stage 1."

Call such description medium level: says how the TM works but not as explicit as a state-diagram.

- **Example TM:** Consider a TM for the language $A = \{0^{2^n} \mid n \geq 0\}$

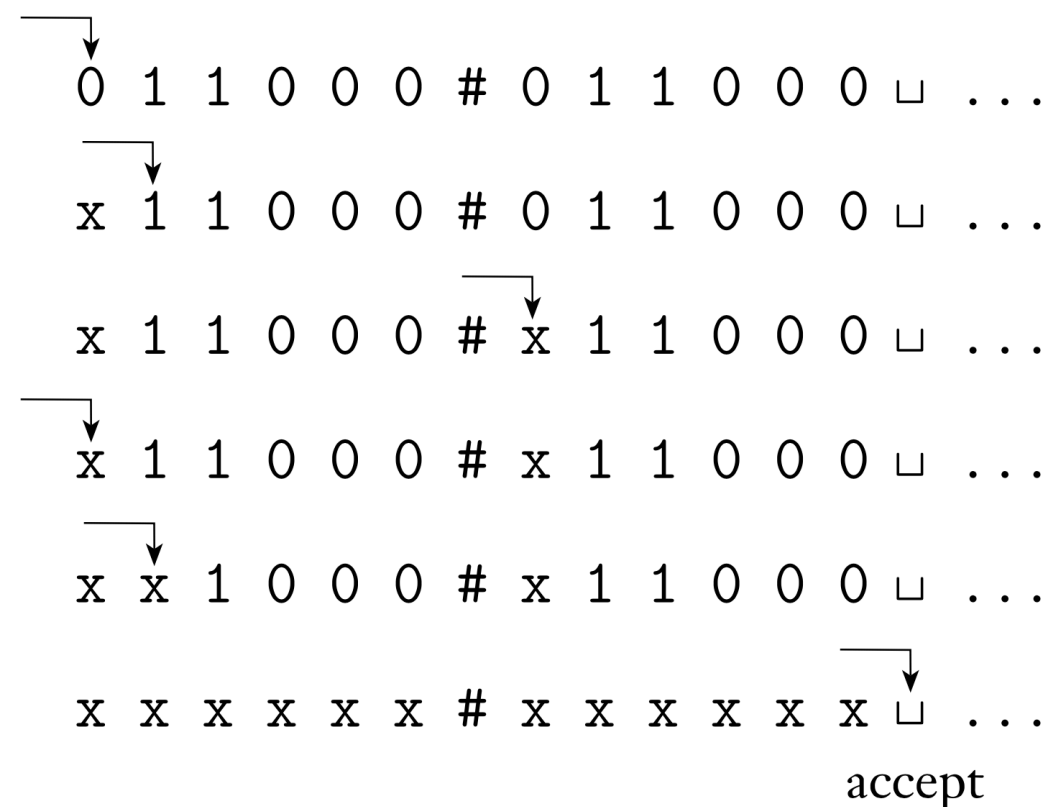
Each transition of the form $x \rightarrow y, D$ means “upon reading x , replace it with symbol y and move the tape head in direction D ”. If y is omitted x is left unchanged



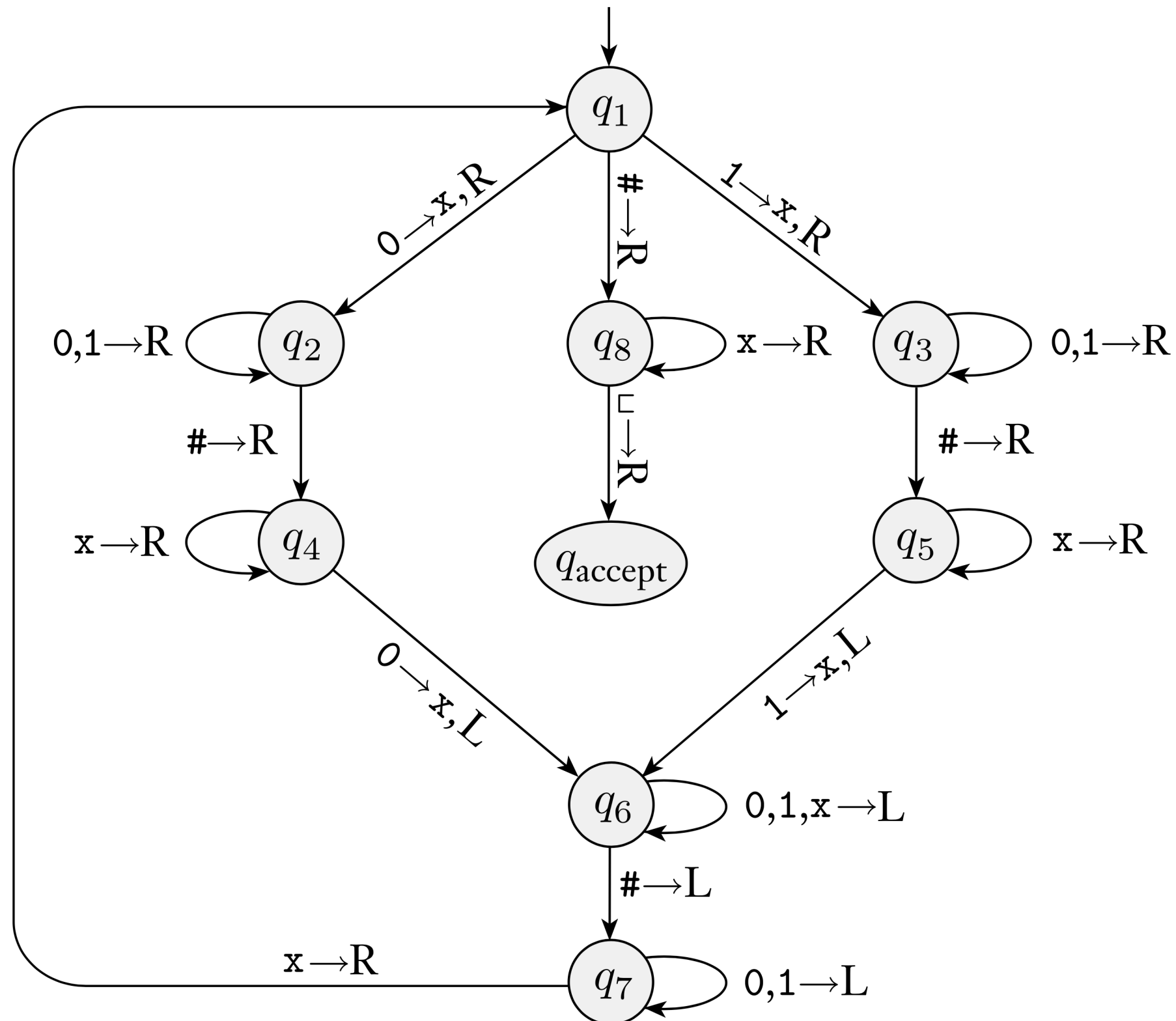
- **Example TM:** Consider a TM for the language $A = \{w\#w \mid w \in \{0,1\}^*\}$

$M =$ "On input string w ,

1. Zig-zag across the tape on either side of $\#$ to check if these positions contain the same symbol. If they mismatch or no $\#$ is found, **reject**.
2. When all symbols on left of $\#$ have been crossed off, if there are any remaining symbols on right of $\#$, **reject**. Otherwise **accept**.



- **Example TM:** Consider a TM for the language $A = \{w\#w \mid w \in \{0,1\}^*\}$



Levels of Description

- **Low-level description** using δ and state diagram provides a complete picture but quickly become unwieldy
- Stick to "**medium-level**" description from now on
 - Describes how the TM works in English
 - What is OK: can include anything in a high-level description, as long as you are convinced that, if you had to, you could design a (low-level) Turing machine for it!
- Will move on to high-level descriptions (algorithms) as we progress

Practice

- **Exercise.** Give a medium-level description of a TM that recognizes $L = \{a^n b^n c^n \mid n \geq 0\}$

Exercise. Give a medium-level description of a TM that recognizes $L = \{a^n b^n c^n \mid n \geq 0\}$

Solution.

In the following we assume that tape symbols $x, y, z \notin \Sigma$

M = “On input string w :

1. Scan the input from left to right to check if it is of the form $a^*b^*c^*$ and reject if it is not.
2. Return head to left hand end of tape.
3. While there are a's remaining on the tape, do:
 - Replace the first a with an x, scan right until a b occurs; replace it with y, and scan to the right until a c occurs; replace it with z. If the corresponding b and c for each a are not found, reject.
4. If there are no b's or c's remaining on the tape, accept. Otherwise, reject.

Why Study Turing Machines

- Not a good model to think about *fast* computation
- Fast algorithms are a subject of CS 256
- In this class, we are interested in finding out if we can solve a problem at all
- To show a problem is not solvable, we need a model of what it means to solve a problem
 - Church-Turing Thesis:

*Intuitive notion
of algorithms*

equals

*Turing machine
algorithms*